

Spec — Lesen Preview: Translate Hover via MyMemory

mini-moi · Guild Created: 2026-06-14 — Claude.ai Status: spec_ready For: Claude Code
Depends on: spec_security_architecture_2026-06-14.md (Cloudflare Pages migration must be complete — this fixes a breakage that only becomes permanent after that migration)

Problem

The Mein Deutsch Lesen preview has a word-hover feature that calls `/app/german/api/translate` for German→English translations via DeepL. In the current preview, this fails because preview visitors aren't authenticated — they get an HTML login redirect instead of JSON, causing a JS parse error.

After the Cloudflare Pages migration, the problem is permanent: no Flask backend is reachable from `minimoi.ai` at all.

Decision: Option A — MyMemory public API

Redirect translate calls in the fetch intercept to MyMemory (`api.mymemory.translated.net`). Free, no auth required, client-side — works in the static Cloudflare Pages context. Quality is good enough for German→English word/phrase translation in a demo context.

Options B (pre-captured cache — gets large, rebuilds every refresh) and C (silent fail — breaks the best interactive demo element in Lesen) are not worth the tradeoffs.

Step 0 — Confirm before writing the intercept (Claude Code)

One unknown before any code is written:

Check the `/app/german/api/translate` Flask route and the `lesen.html` JS that calls it.
Confirm:

1. **Exact POST body shape** — what field name carries the phrase? e.g. `{phrase: "Kaffee"}` or `{word: "Kaffee"}` or `{text: "Kaffee"}`
2. **Exact response shape** the Lesen JS reads — what field name(s) does the JS use from the Flask response? e.g. `data.translation` or `data.result` or `data.translatedText`

Fill these into the `(FIELD_NAME_*)` placeholders in the intercept code below before committing.

MyMemory API reference

```
GET https://api.mymemory.translated.net/get?q={phrase}&langpair=de|en
```

No auth required. Response shape:

```
json
{
  "responseData": {
    "translatedText": "Coffee",
    "match": 1
  },
  "responseStatus": 200
}
```

Rate limits: 1,000 words/day per IP without a key — sufficient for a demo preview with occasional visitors (hover translations are on-demand, not pre-loaded). No API key needed for this use case.

Fetch intercept code block

Drop into `(FETCH_INTERCEPT_JS)` in `(capture_snapshot.py)`. Replace `(FIELD_NAME_IN_POST_BODY)` and `(FIELD_NAME_IN_RESPONSE)` with confirmed values from Step 0.

```
javascript
```

```
(function() {
  const _originalFetch = window.fetch;

  window.fetch = function(url, options) {
    // Intercept translate API calls only
    if (typeof url === 'string' && url.includes('/api/translate')) {
      return (async () => {
        try {
          // Extract phrase from POST body
          const body = options && options.body
            ? JSON.parse(options.body)
            : {};
          const phrase = body.FIELD_NAME_IN_POST_BODY || '';

          if (!phrase) {
            return new Response(
              JSON.stringify({ FIELD_NAME_IN_RESPONSE: '' }),
              { status: 200, headers: { 'Content-Type': 'application/json' } }
            );
          }

          // Call MyMemory
          const mmUrl = 'https://api.mymemory.translated.net/get'
            + '?q=' + encodeURIComponent(phrase)
            + '&langpair=de|en';

          const mmRes = await _originalFetch(mmUrl);
          const mmData = await mmRes.json();
          const translation = (
            mmData.responseData && mmData.responseData.translatedText
          ) || '';

          // Return response in the shape Lesen JS expects
          return new Response(
            JSON.stringify({ FIELD_NAME_IN_RESPONSE: translation }),
            { status: 200, headers: { 'Content-Type': 'application/json' } }
          );
        } catch (err) {
          // Silent fail – popover shows empty rather than crashing
          console.warn('[preview] translate intercept error:', err);
          return new Response(
            JSON.stringify({ FIELD_NAME_IN_RESPONSE: '' }),

```

```
        { status: 200, headers: { 'Content-Type': 'application/json' } }
      );
    }
  })();
}

// All other fetch calls pass through unchanged
return _originalFetch.apply(this, arguments);
};
})();
```

Where it goes: `capture_snapshot.py` injects `FETCH_INTERCEPT_JS` into every captured preview page. This intercept should be added to the existing block, not as a separate injection — one `<script>` tag per page.

Definition of Done

- ☐ Step 0 complete — POST body field name and response field name confirmed
 - ☐ Placeholders replaced with confirmed field names
 - ☐ Intercept added to `FETCH_INTERCEPT_JS` in `capture_snapshot.py`
 - ☐ Snapshot regenerated (re-run `capture_snapshot.py`)
 - ☐ Manual test in browser (incognito, no auth): hover a word in Lesen preview → popover shows English translation → no JS console errors
 - ☐ Test with a multi-word phrase (some Lesen hovers may be phrases, not single words — confirm MyMemory handles both)
 - ☐ Robert confirms hover feels correct in the live preview
-

Commit

```
bash
```

```
git add tools/capture_snapshot.py static/public/preview/german/lesen.html
git commit -m "fix: Lesen preview translate hover – route to MyMemory"
```

Fetch intercept in preview redirects /api/translate calls to MyMemory public API (de→en, no auth). Fixes JS parse error caused by HTML login redirect in preview context; permanent fix for post-Cloudflare-Pages migration where no Flask backend is reachable. Silent fail on error (empty popover, no crash)."

```
git push origin main
```